

6/5

사실 LLM은 우리가 생각했던 것보다 더 복잡한 구조를 가지고 있음.

→ 더 세분화되어 쪼개져야 함. Decompose AI !!

Workflow:

Where to? (들어온 정보를 어디로 보낼지 결정, 분배) $\Leftarrow \Rightarrow$ **Authentication, legacy, FAQ** 등 $\Rightarrow$

Verboser

- 시간, 돈이 많이 듦 (사람이 직접 하니까)
- But 그만큼 완성도가 높음.

각 'Agent' = LLM 하나

- 정답이 따로 있는 게 x (말해보니 잘 못 할 수도) – 설계는 사람이 직접.
- Agent가 붙지 않았으면 AI가 굳이 필요 x, 이미 존재하는 '시스템' 자체를 의미.
- FDE (forward deployed engineering; 전방 배치 기술자) 많이 보유하는 회사: 팔란티어

*여기에 LLM(planner) 이 들어오면 (Agentic AI)?

- 입력이 바로 planner로 흘러들어감 (planner는 workflow 안에 속하지 x)
- 이후 $\Rightarrow$ Executor (workflow)에게 보냄
- Workflow에 관한 걸 설계하는 역할 (AI가 설계를 완전히 담당, 사람의 개입을 배제)
- 마지막에 LLM(evaluator)이 평가.
- 만약 마음에 안 들면 다시 planner로 돌아가거나 (설계 자체가 문제 되었을 때), executor 로 돌아가 (설계는 잘 됐지만, 실행에 문제가 있을 때) = 'loop'
- 그러나 loop를 많이 돌린다고, 무조건 만족할 만한 결과가 나오는 것은 x.
실행을 돌리기 어려운 실수를 할 수도.

> AI가 모든 것을 혼자 알아서 설계한다고 생각하면 안 된다는 점이 중요하게 느껴졌다. Agent가 아무리 똑똑해도, 각각의 Agent가 어떤 역할을 하고 어떤 순서로 작동해야 하는지는 아직 사람이 설계해야 한다. 특히 FDE처럼 기술을 현실의 문제와 연결하는 역할이 중요하다는 점에서, AI 시대에도 사람의 기획력과 문제 이해 능력이 필요하다고 느꼈다.

<AI 관련 용어>

- RAG (Retrieval-Augmented Generation)
 - LLM이 똑똑하지 않아서 등장하게 된 용어
 - Retrieve (벡터에서 가져와서) $\rightarrow$ Augment (chunk에 더함) $\rightarrow$ Generate (내가 원하는 답변을 만들어냄)
 - 그러나 헛소리 할 가능성이 높음

When 자료를 다 때려넣어서 질문을 하고 싶은데, 자료 양이 너무 클 때:

- 모든 입력 chunk들을 벡터로 만듦
- 사용 LLM = 벡터화 하는 역할
- 제일 비슷한 벡터 하나를 고름 (나의 질문에 대한 답변이 그곳에 담겨있을 것)
- 거기에 해당되는 chunk를 찾아서 벡터와 붙임
 $\rightarrow$ 이를 LLM에 입력

⇒ 그러면 이제 내 질문에 잘 답변을 할 것!

➢ AI가 모든 자료를 다 읽는 것이 아니라, 관련성이 높은 정보만 선택해서 사용한다는 점이 흥미로웠다.

- **Agentic AI**: 사람이 모든 단계를 지시하지 않아도, AI가 스스로 목표를 달성하기 위한 계획을 설계하고, 자율적으로 업무 수행.
- **Hallucination**: 환각 현상, 헛소리하는 현상
- **LLM 고도화 기술**
 1. **Pretraining**
 - 알갱이를 LLM을 통해 처리
 - But 알갱이로 단어를 쓰지 x. 마음대로 더 작은/큰 단위로 자르기도 함. = **'Token'** (훈련할 때의 단위)
 - 요즘은 어떤 사람의 역량을 token의 양으로 측정, 평가하기도 함.
 2. **Fine-tuning**
 - 질문과 답변의 형태를 잘하는 버전으로 바꾸기.
 - 반드시 **Q-A set**로 되어있는 데이터만 사용해야 함.
= Pretraining과 훈련방식 자체는 비슷하지만, 들어가는 데이터 형식이 다름.
 - 모델 자체의 가중치 (행렬)을 바꾸는 기술
 3. **강화**
 - 상업화 이후 예기치 못한 상황 (ex. 성적, 비윤리적인 답변을 하는 등)을 미리 막아두는 것
 - **Reinforcement learning by human feedback** (당근과 채찍)
- **Embedding**
 - 벡터: embedding vector
 - 인간의 언어/이미지를 AI가 계산할 수 있도록 벡터화 하는 것
- **Context Window**: AI가 한번에 처리할 수 있는 입력의 최대 용량
- **과적합 (overfitting)**: 훈련 데이터에만 너무 과하게 적합하도록 학습되어, 실제 새로운 데이터/실전 업무에 사용했을 때 예측 능력이 떨어지는 현상
- **Project manager (PM)**: 프로젝트가 문제없이 완수되도록 하는 사람
- **API** = 부품 (이후 개발에 사용할 수 있는)
- **클라이언트**: 서비스를 요청하는 사용자 기기
- **서버**: 그 요청을 받아 데이터를 처리하고 저장해주는 대형 컴퓨터
- **클라우드 컴퓨팅**: 회사 내부에 비싼 컴퓨터 서버를 직접 두고 관리하는 대신, 빅테크 기업의 대형 데이터 센터 공간을 필요한 만큼 인터넷으로 빌려쓰는 서비스
→ 컴퓨터에 대한 사용료만 내면 됨

서비스 형태 및 데이터베이스

- **SaaS (Software as a Service)**: 컴퓨터 뿐만 아니라 서비스도 빌려줌
- **PasS (Platform as a Service)**
- **IaaS (Infrastructure as a Service)**
- **온프레미스**: 자체 서버에서 클라우드로의 전환 과정

- Front-end: 사용자가 직접 보고 클릭하는 화면을 개발 (언어: Javascript 등)
- Back-end: 화면 뒤의 데이터 처리, 서버 운영
- Agile 방법론: 짧은 주기로 대충 만들어보고, 점점 맞춰나가는 개발 방식
- MVP (Minimum Viable Product): 시제품
 - B2B (business to business), B2C (cloud에 띄워두고 쓰는지 안 쓰는지 확인)
 - 둘 중 뭐를 사용하든, 일단 고객의 반응을 살피며 간은 봐야 하니까 MVP를 사용
- UI: 화면 디자인
- UX: 나의 체감; 사용자가 앱을 쓰면서 느끼는 편리함, 동선, 만족도 등 종합적인 경험
- Legacy: 회사에 있던 기존 시스템
- Domain: 특정 도메인을 가리킬 때
- 정형: 엑셀에 들어갈 수 있는 숫자 데이터
- 비정형 데이터: text, image
- 토큰 가성비: 할당된 토큰을 얼마나 잘 썼는지 확인
- 지연시간 (Latency): 사용자가 입력을 넣고 첫 번째 답변을 출력할 때까지 걸리는 시간
- Human-in-the-Loop: 인간의 승인, 컨펌을 거치도록 설계 (지금 우리가 사용하는 방식)
 - 이것의 반대가 Agentic AI
- 가드레일: AI 모델이 기업의 정책을 벗어나거나, 편향된 발언, hallucination을 일으키지 않도록 입력, 출력 단계에 씌우는 안전 필터 및 규칙
 - ex) 욕설 x, 시스템 프롬프트 유출하지 마라

*타이밍, 집요!!!! *Tenacious*

➤ 오늘 수업을 통해 지금까지는 다소 생소하게 느껴졌던 AI 관련 용어들을 한 번에 정리할 수 있었다. 처음에는 어렵게 느껴졌지만, 각각의 개념이 실제 AI 서비스 구조와 연결된다는 점을 알게 되면서 AI를 더 구체적으로 이해할 수 있다는 자신감이 생겼다. 특히, 이번 학기 수업을 통해 AI와 직접적으로 관련된 전공이 아니더라도 이러한 지식을 배우는 것이 앞으로 더욱 중요해질 것이라고 느꼈다. 앞으로도 모르는 개념을 그냥 넘기지 않고, 집요하게 질문하고 배워가며 나의 역량과 경쟁력을 키워나가고 싶다.